MIT | Arts, Commerce & Science College

Alandi (D), Pune – 412105

Department of Computer Application

F. Y. M.Sc. (Computer Application) Semester-II

Lab Course –2413MJCP205

Lab course based on Introduction to Artificial Intelligence & Machine Learning

Lab Book

Name: _____

Roll No.:_____ Division: _____

Academic Year:_____

Editors:

1) Prof. Omkar K. Dengi, Assistant Professor, MIT Arts, Commerce and Science College, Alandi, Pune.

Reviewed By:

1) Prof. Harsha Patil, Assistant Professor, MIT Arts, Commerce and Science College, Alandi, Pune.

Introduction

1. **About the work book:**
   This Lab book is intended to be used by F.Y.MSc. (Computer Application) students for the Introduction to Artificial Intelligence and Machine Learning in Semester–II. This Lab book is designed by considering all the practical concepts / topics mentioned in the syllabus.

2. **The objectives of this workbook are:**

   1) Define the scope and significance of Artificial Intelligence and Machine Learning.
   2) Equip students with skills to analyze data effectively, including statistical measures, data cleaning, and visualization techniques.
   3) Encourage students to develop critical thinking and problem-solving skills by engaging with various algorithms and techniques in AI and ML.
   4) Introduce students to popular machine learning libraries (e.g., Scikit-learn, Pandas) and their functionalities through practical assignments.
   5) Offer hands-on exercises that allow students to apply theoretical knowledge to real world problems using programming languages like python.

3. **How to use this workbook:**
   The Lab book Contains Practical Questions related to introduction to Artificial Intelligence and Machine Learning. It is divided into assignments. Each assignment has three SETs. It is mandatory for students to complete the SET A and SET B in given slot.

3.1 Instructions to the students

Please read the following instructions carefully and follow them:

1) Students are expected to carry this book every time they come to the lab for practice.
2) Students should prepare oneself beforehand for the Assignment by reading the relevant material.
3) Instructor will specify which problems to solve in the lab during the allotted slot and the student should complete them and get verified by the instructor. However, student should spend additional hours in Lab and at home to cover as many problems as possible given in this Lab book.
4) Students will be assessed for each exercise on a scale from 0 to 5.

Not done	0
Incomplete	1
Late Complete	2
Needs improvement	3
Complete	4
Well Done	5

3.2 Instruction to the Instructors

1) Explain the assignment and related concepts in around ten minutes using a whiteboard if required or by demonstrating the software.
2) Instructors should evaluate each assignment carried out by a student on a scale of 5 as specified above by ticking the appropriate box.
3) The value should also be entered on the assignment completion page of the respective Lab course.

3.3 Instructions to the Lab Administrator

1) You have to ensure appropriate hardware and software is made available to each student.
2) The operating system and software requirements on server side and also client-side areas given below:
   - Server and Client Side -(Operating System) Linux/Windows
   - Jupyter /Spyder notebook or Anaconda Navigator or any IDLE of python on any Platform.

Table of Contents

Assignment No	Name of Assignment	Page No
Assignment No.1	Outline of AI	1 - 5
Assignment No.2	Solving Problem by Searching	6 -15
Assignment No.3	Introduction to Machine Learning	16 – 25
Assignment No.4	Basics of Data and Statistics	26 – 31
Assignment No.5	Supervised Learning Model	32 – 44
Assignment No. 6	Unsupervised Learning Model	45 - 54

Assignment Completion Sheet

Lab Course			
Introduction to Artificial Intelligence and Machine Learning			
Sr. No	**Assignment Name**	**Marks (out of 5)**	**Teachers Sign**
1	Outline of AI		
2	Solving Problem by Searching		
3	Introduction to Machine Learning		
4	Basics of Data and Statistics		
5	Supervised Learning Model		
6	Unsupervised Learning Model		
Total (Out of 30)			
Total (Out of 20)			

This is to certify that Mr./ Ms. _____has successfully completed the Introduction of Artificial Intelligence and Machine Learning course work for Lab Course- 2413MJCP205 and has scored _____Marks out of 20.

Instructor **H.O.D. / Coordinator**

Internal Examiner **External Examiner**

Assignment No 1
Outline of AI

Objectives:

Students will be able to

- Learn about the basic concepts of Artificial Intelligence, including its definitions, history, and current state.
- Understand the structure and behavior of intelligent agents and their interaction with environments.
- Examine various applications of AI in real-world scenarios.

Prerequisites:

- ✓ Basic understanding of programming concepts.
- ✓ Familiarity with Python is beneficial for implementing AI algorithms.

Introduction:

In today's world, technology is growing very fast, and we are getting in touch with different new technologies day by day. Here, one of the booming technologies of computer science is Artificial Intelligence which is ready to create a new revolution in the world by making intelligent machines. Artificial Intelligence is now all around us. It is currently working with a variety of subfields, ranging from general to specific, such as self-driving cars, playing chess, proving theorems, playing music, Painting, etc. AI is one of the fascinating and universal fields of Computer science which has a great scope in future. AI holds a tendency to cause a machine to work as a human.

What is Artificial Intelligence?

Artificial Intelligence is composed of two words Artificial and Intelligence, where Artificial defines "man-made," and intelligence defines "thinking power", hence AI means "a man-made thinking power."

In today's rapidly advancing technological landscape, AI has become a household term. From chatbots and virtual assistants to self-driving cars and recommendation algorithms, the impact of AI is ubiquitous. But what exactly is AI and how does it work?

At its core, Artificial Intelligence refers to the simulation of human intelligence in machines that are programmed to think, reason, and learn like humans. Rather than being explicitly programmed for specific tasks, AI systems use algorithms and vast amounts of data to recognize patterns, make decisions, and improve their performance over time. Artificial Intelligence encompasses a wide range of technologies, including machine learning, natural language processing, computer vision, and robotics. These technologies enable AI systems to perform complex tasks, such as speech recognition and face detection, with remarkable accuracy.

Intelligent system:

Intelligent systems can also include sophisticated AI-based software systems, such as chatbots, expert systems and other types of software. Essentially, an intelligent device is anything that contains a functional, although not usually general-purpose, computer with Internet connectivity. An embedded system may be powerful and capable of complex processing and data analysis, but it is

usually specialized for tasks relevant to the host machine. Intelligent systems exist all around us in point-of-sale (POS) terminals, digital televisions, traffic lights, smart meters, automobiles, digital signage and airplane controls, among a great number of other possibilities. Built-in intelligence is an integral component of the developing internet of things (IoT), in which almost everything imaginable can be provided with unique identifiers and the ability to automatically transfer data over a network without requiring human-to-human or human-to-computer interaction.

Foundations of AI and sub areas of AI:

AI into five distinct research areas

- Machine Learning
- Expert Systems
- Computer Vision
- Natural Language Processing
- Robotics

Machine Learning:

Machine learning (ML) is the science of empowering machines to make decisions without human intervention. This sub-discipline forms the backbone of AI, enabling computers to learn and interpret patterns in images, sounds, and structured data using multidimensional arrays. ML is further subdivided into four types of learning:

Supervised learning: Given an array of features (i.e., week of the year, price, etc.) and a labeled output variable (e.g., sales), predict the best possible estimate of the label variable given some new input array.

Unsupervised learning: Given an array of features (e.g., demographic information, ZIP code, etc.), expose and visualize hidden relationships and anomalies within the array.

Semi-supervised: Given an array of features and a limited quantity of some labeled output variable, predict the best possible estimates for the missing label variables.

Reinforcement learning: Given some objective, train an artificial agent to maximize its utility according to some user-defined utility function.

Expert Systems:

An expert system (ES) is an artificial agent which leverages pre-programmed knowledge to offer advice or make decisions. An expert system can take advantage of human insights discovered through trial and error

- Expert systems are more predictable and are less likely to make extreme errors when faced with previously-unseen inputs

- Expert systems have historically been faster and easier to implement, though ML has become much more accessible in recent years

Computer Vision:

Computer vision (CV) is the automatic extraction, analysis, and interpretation of images or videos. CV converts photos and videos into numerical arrays, enabling ML algorithms to draw inferences, make predictions, and even generate new images based on user-defined inputs.

Natural Language Processing:

Natural language processing (NLP) is the automatic extraction, analysis, and generation of human language. NLP algorithms parse sentences in various ways (e.g., splitting by word, splitting by letter, reading both left-to-right and right-to-left, etc.) to automatically draw inferences about the writer's meaning and intent.

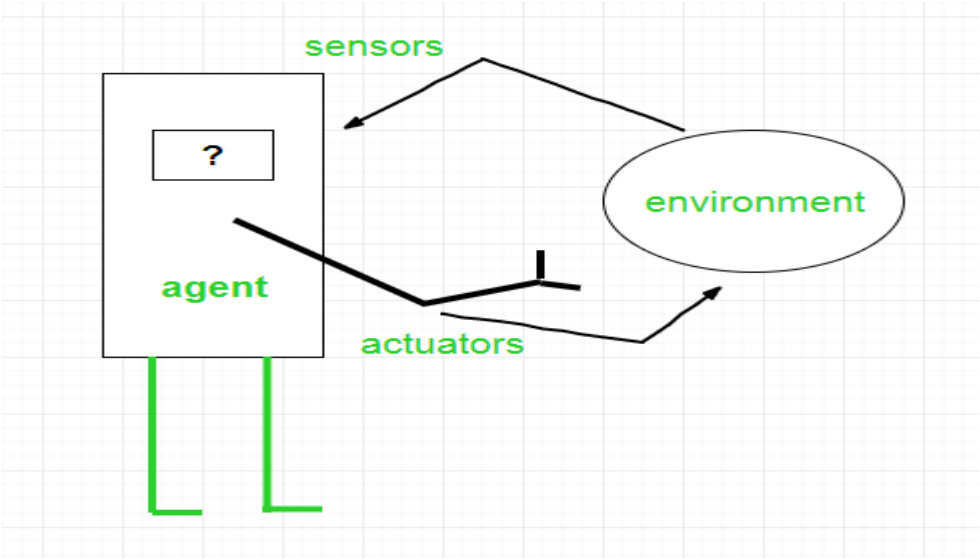
Robotics:

Robotics is the science of designing, constructing, operating, and applying robots to solve human problems. Robots come in thousands of shapes and sizes, making it difficult to nail down the precise meaning of the term. The field of robotics research has evolved at breakneck speed over the past decade. Most robots rely on expert systems to accomplish their objective, but the robots of tomorrow will become exponentially more useful by incorporating machine learning, computer vision, and natural language processing into their operating systems.

AI Agents:

In artificial intelligence, an agent is a computer program or system that is designed to perceive its environment, make decisions and take actions to achieve a specific goal or set of goals. The agent operates autonomously, meaning it is not directly controlled by a human operator.

Agents can be classified into different types based on their characteristics, such as whether they are reactive or proactive, whether they have a fixed or dynamic environment, and whether they are single or multi-agent systems.



Interaction of Agents with the Environment

Artificial intelligence is defined as the study of rational agents. A rational agent could be anything that makes decisions, such as a person, firm, machine, or software. It carries out an action with the best outcome after considering past and current percepts (agent's perceptual inputs at a given instance). An AI system is composed of an **agent and its environment**. The agents act in their environment. The environment may contain other agents.
An agent is anything that can be viewed as:

- Perceiving its environment through **sensors** and
- Acting upon that environment through **actuators**.

Lab Assignments

SET A

1. Write a Python Program to Create a Simple Chabot.
2. Write a 1-page summary covering key milestones in AI development.

SET B

1. Write a Report with examples to Analyze AI Applications in Different Industries. (Healthcare, Finance, and Transportation.).
2. Write a Python Program to Create a Flowchart of AI Decision-Making Process.

SET C

1. Create a presentation on intelligent agents.
   (Include definitions, types of agents, and examples of intelligent behavior.)
2. Write a Python Program to Implement a Simple AI Agent

Assignment Evaluation

0: Not Done [ ]	1: Incomplete [ ]	2: Late Complete [ ]
3: Need Improvement [ ]	4: Complete [ ]	5: Well Done [ ]

Signature of Teacher

Assignment No 2
Solving Problem by Searching

Objectives:

Students will be able to

- To Study uninformed (blind) and informed (heuristic) search strategies used in problem-solving.
- Gain hands-on experience by implementing various search algorithms like DFS, BFS, and A*.
- To understand how to evaluate the effectiveness of different search methods in finding solutions.

Prerequisites:

- ✓ Knowledge of data structures (e.g., graphs, trees).
- ✓ Basic programming skills in Python to implement search algorithms.

Introduction:

Search algorithms in AI are the algorithms that are created to aid the searchers in getting the right solution. The search issue contains search space, first start and end point. Now by performing simulation of scenarios and alternatives, searching algorithms help AI agents find the optimal state for the task. Logic used in algorithms processes the initial state and tries to get the expected state as the solution. Because of this, AI machines and applications just functioning using search engines and solutions that come from these algorithms can only be as effective as the algorithms.

AI agents can make the AI interfaces usable without any software literacy. The agents that carry out such activities do so with the aim of reaching an end goal and develop action plans that in the end will bring the mission to an end. Completion of the action is gained after the steps of these different actions. The AI-agents finds the best way through the process by evaluating all the alternatives which are present. Search systems are a common task in artificial intelligence by which you are going to find the optimum solution for the AI agents.

Problem-solving agents:

In Artificial Intelligence, Search techniques are universal problem-solving methods. **Rational agents** or **Problem-solving agents** in AI mostly used these search strategies or algorithms to solve a specific problem and provide the best result. Problem-solving agents are the goal-based agents and use atomic representation. In this topic, we will learn various problem-solving search algorithms.

Importance of Search Algorithms in Artificial Intelligence:

Here are some important factors of the role of search algorithms using AI are as follows.

1. Solving problems:

"Workflow" logical search methods like describing the issue, getting the necessary steps together, and specifying an area to search help AI search algorithms get better in solving problems. Take for instance the development of AI search algorithms which support applications like Google Maps by finding the fastest way or shortest route between given destinations. These programs basically conduct the search through various options to find the best solution possible.

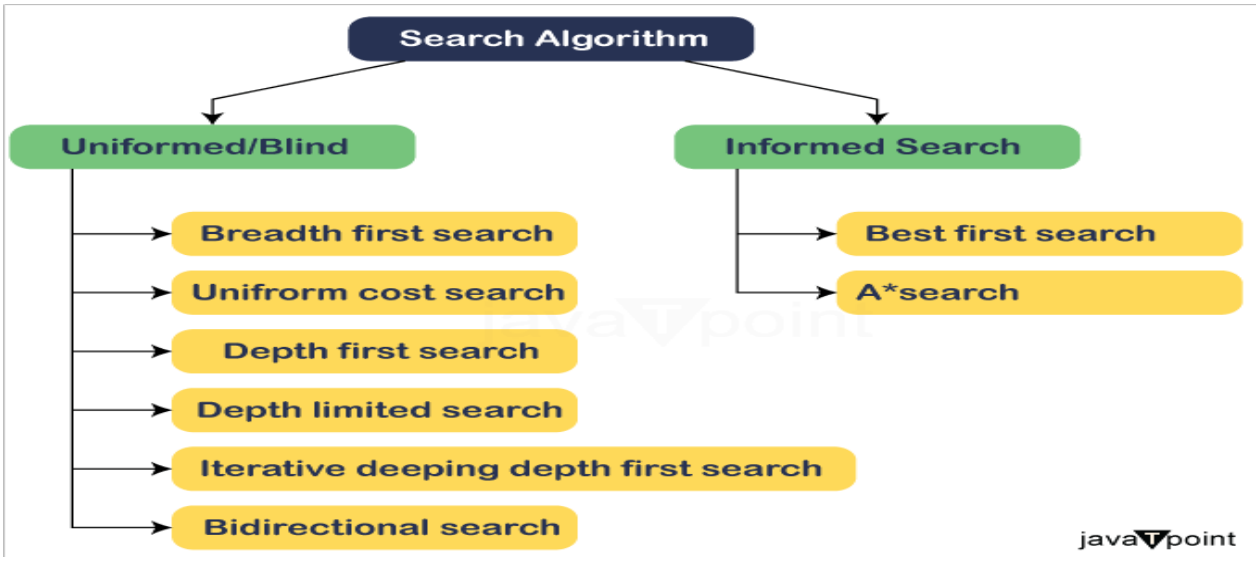
2. Search programming: Many AI functions can be designed as search oscillations, which thus specify what to look for in formulating the solution of the given problem.

3. Goal-based agents: Instead, the goal-directed and high-performance systems use a wide range of search algorithms to improve the efficiency of AI. Though they are not robots, these agents look for the ideal route for action dispersion so as to avoid the most impacting steps that can be used to solve a problem. It is their main aims to come up with an optimal solution which takes into account all possible factors.

4. Support production systems: AI Algorithms in search engines for systems manufacturing help them run faster. These programmable systems assist AI applications with applying rules and methods, thus making an effective implementation possible. Production systems involve learning of artificial intelligence systems and their search for canned rules that lead to the wanted action.

5. Neural network systems: Beyond this, employing neural network algorithms is also of importance to the neural network systems. The systems are composed of these structures: a hidden layer, and an input layer, an output layer, and nodes that are interconnected. One of the most important functions offered by neural networks is to address the challenges of AI within any given scenario. AI is somehow able to navigate the search space to find the connection weights that will be required in the mapping of inputs to outputs. This is made better by search algorithms in AI.

Types of search algorithms:
Based on the search problems we can classify the search algorithms into uninformed (Blind search) search and informed search (Heuristic search) algorithms.



Uninformed/Blind Search:
The uninformed search does not contain any domain knowledge such as closeness, the location of the goal. It operates in a brute-force way as it only includes information about how to traverse the tree and how to identify leaf and goal nodes. Uninformed search applies a way in which a search tree is searched without any information about the search space like initial state operators and tests for the goal, so it is also called blind search. It examines each node of the tree until it achieves the goal node.

It can be divided into six main types:

- Breadth-first search
- Uniform cost search
- Depth-first search
- Depth limited search
- Iterative deepening depth-first search
- Bidirectional Search

1. Breadth-first Search:

- Breadth-first search is the most common search strategy for traversing a tree or graph. This algorithm searches breadth wise in a tree or graph, so it is called breadth-first search.
- BFS algorithm starts searching from the root node of the tree and expands all successor nodes at the current level before moving to nodes of next level.
- The breadth-first search algorithm is an example of a general-graph search algorithm.
- Breadth-first search implemented using FIFO queue data structure.

Informed Search:
Informed search algorithms use domain knowledge. In an informed search, problem information is available which can guide the search. Informed search strategies can find a solution more efficiently than an uninformed search strategy. Informed search is also called a Heuristic search. A heuristic is a way which might not always be guaranteed for best solutions

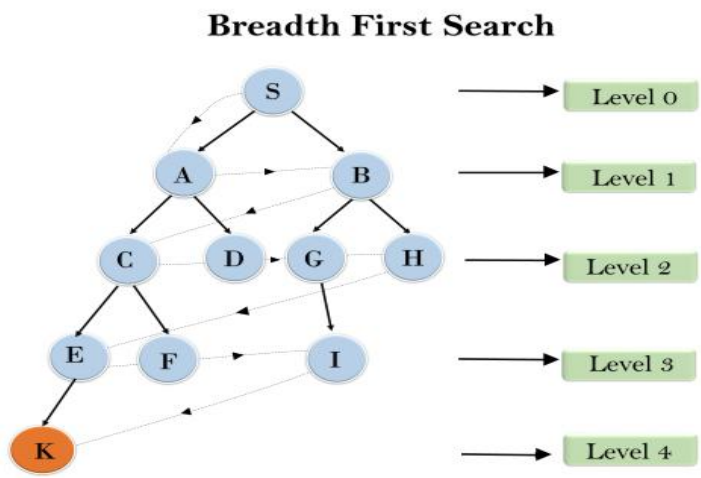
but guaranteed to find a good solution in reasonable time. Informed search can solve many complex problems which could not be solved in another way.

An example of informed search algorithms is a traveling salesman problem.

1. Greedy Search
2. A* Search

Example:
In the below tree structure, we have shown the traversing of the tree using BFS algorithm from the root node S to goal node K. BFS search algorithm traverse in layers, so it will follow the path which is shown by the dotted arrow, and the traversed path will be:

1. S---> A--->B---->C--->D---->G--->H--->E---->F---->I---->K



Time Complexity: Time Complexity of BFS algorithm can be obtained by the number of nodes traversed in BFS until the shallowest Node. Where the d= depth of shallowest solution and b is a node at every state.

$T(b) = 1+b^2+b^3+.....+ b^d = O(b^d)$

Space Complexity: Space complexity of BFS algorithm is given by the Memory size of frontier which is $O(b^d)$.

Completeness: BFS is complete, which means if the shallowest goal node is at some finite depth, then BFS will find a solution.

Optimality: BFS is optimal if path cost is a non-decreasing function of the depth of the node.

2. Depth-first Search:

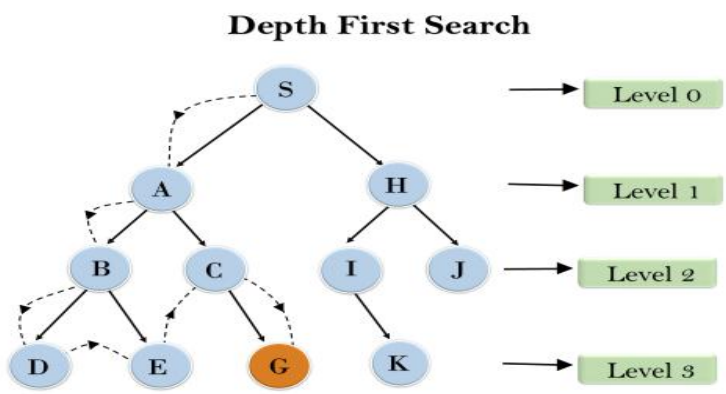
- Depth-first search isa recursive algorithm for traversing a tree or graph data structure.

- It is called the depth-first search because it starts from the root node and follows each path to its greatest depth node before moving to the next path.
- DFS uses a stack data structure for its implementation.
- The process of the DFS algorithm is similar to the BFS algorithm.

Example:
In the below search tree, we have shown the flow of depth-first search, and it will follow the order as:

Root node--->Left node ----> right node.

It will start searching from root node S, and traverse A, then B, then D and E, after traversing E, it will backtrack the tree as E has no other successor and still goal node is not found. After backtracking it will traverse node C and then G, and here it will terminate as it found goal node.



Completeness: DFS search algorithm is complete within finite state space as it will expand every node within a limited search tree.

Time Complexity: Time complexity of DFS will be equivalent to the node traversed by the algorithm. It is given by:

$T(n)= 1+ n^2+ n^3 +.....+ n^m=O(n^m)$

Where, m= maximum depth of any node and this can be much larger than d (Shallowest solution depth)

Space Complexity: DFS algorithm needs to store only single path from the root node, hence space complexity of DFS is equivalent to the size of the fringe set, which is **O(bm)**.

Optimal: DFS search algorithm is non-optimal, as it may generate a large number of steps or high cost to reach to the goal node.

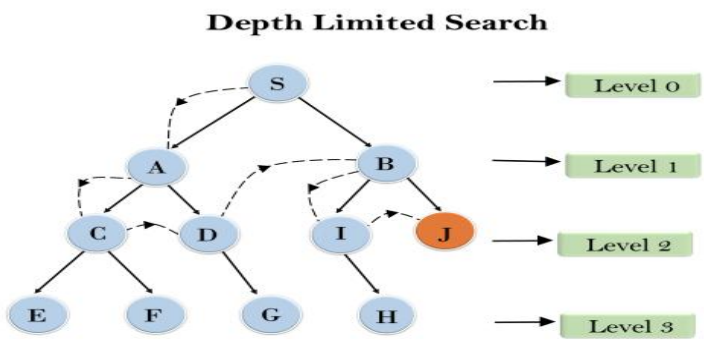
3. Depth-Limited Search Algorithm:

A depth-limited search algorithm is similar to depth-first search with a predetermined limit. Depth-limited search can solve the drawback of the infinite path in the Depth-first search. In this algorithm, the node at the depth limit will treat as it has no successor nodes further.

Depth-limited search can be terminated with two Conditions of failure:

- **Standard failure value:** It indicates that problem does not have any solution.
- **Cutoff failure value:** It defines no solution for the problem within a given depth limit.

Example:



Completeness: DLS search algorithm is complete if the solution is above the depth-limit.

Time Complexity: Time complexity of DLS algorithm is $\mathbf{O(b^{\ell})}$ where b is the branching factor of the search tree, and l is the depth limit.

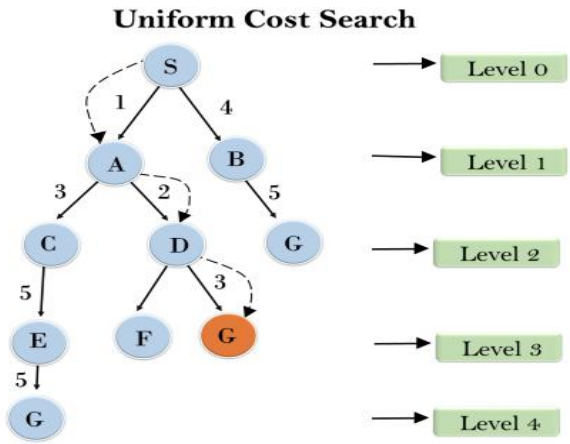
Space Complexity: Space complexity of DLS algorithm is $O(\mathbf{b \times \ell})$ where **b** is the branching factor of the search tree, and **l** is the depth limit.

Optimal: Depth-limited search can be viewed as a special case of DFS, and it is also not optimal even if $\ell > d$.

4. Uniform-cost Search Algorithm:

Uniform-cost search is a searching algorithm used for traversing a weighted tree or graph. This algorithm comes into play when a different cost is available for each edge. The primary goal of the uniform-cost search is to find a path to the goal node which has the lowest cumulative cost. Uniform-cost search expands nodes according to their path costs form the root node. It can be used to solve any graph/tree where the optimal cost is in demand. A uniform-cost search algorithm is implemented by the priority queue. It gives maximum priority to the lowest cumulative cost. Uniform cost search is equivalent to BFS algorithm if the path cost of all edges is the same.

Example:



Completeness:

Uniform-cost search is complete, such as if there is a solution, UCS will find it.

Time Complexity:

Let C* **is Cost of the optimal solution**, and **ε** is each step to get closer to the goal node. Then the number of steps is = C*/ε+1. Here we have taken +1, as we start from state 0 and end to C*/ε.

Hence, the worst-case time complexity of Uniform-cost search is$\mathbf{O(b^{1 + \lceil C^*/\varepsilon \rceil})}$/.

Space Complexity:

The same logic is for space complexity so, the worst-case space complexity of Uniform-cost search is $\mathbf{O(b^{1 + \lceil C^*/\varepsilon \rceil})}$.

Optimal:

Uniform-cost search is always optimal as it only selects a path with the lowest path cost.

5. Iterative deepening depth-first Search:

The iterative deepening algorithm is a combination of DFS and BFS algorithms. This search algorithm finds out the best depth limit and does it by gradually increasing the limit until a goal is found.

This algorithm performs depth-first search up to a certain "depth limit", and it keeps increasing the depth limit after each iteration until the goal node is found.

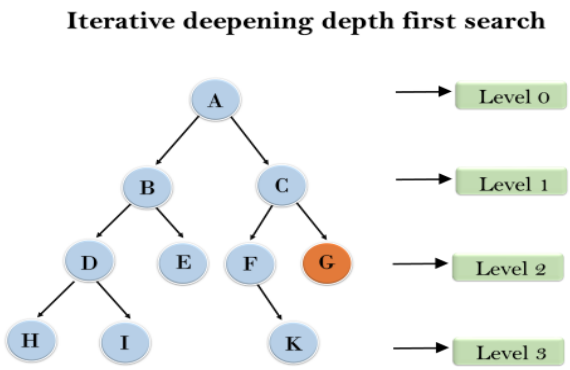
This Search algorithm combines the benefits of Breadth-first search's fast search and depth-first search's memory efficiency.

The iterative search algorithm is useful for uninformed search when the search space is large, and depth of the goal node is unknown.

Here are the steps for Iterative deepening depth first search algorithm:

- Set the depth limit to 0.
- Perform DFS to the depth limit.
- If the goal state is found, return it.
- If the goal state is not found and the maximum depth has not been reached, increment the depth limit and repeat steps 2-4.
- If the goal state is not found and the maximum depth has been reached, terminate the search and return failure.

Example:
Following tree structure shows the iterative deepening depth-first search. The IDDFS algorithm performs various iterations until it does not find the goal node. The iteration performed by the algorithm is given as:



1'st Iteration-----> A
2'nd Iteration----> A, B, C
3'rd Iteration------>A, B, D, E, C, F, G
4'th Iteration------>A, B, D, H, I, E, C, F, K, G
In the fourth iteration, the algorithm will find the goal node.

Completeness:

This algorithm is complete if the branching factor is finite.

Time Complexity:

Let's suppose b is the branching factor and depth is d then the worst-case time complexity is **$O(b^d)$**.

Space Complexity:

The space complexity of IDDFS will be **O(bd)**.

Optimal:

IDDFS algorithm is optimal if path cost is a non- decreasing function of the depth of the node.

6. Bidirectional Search Algorithm:

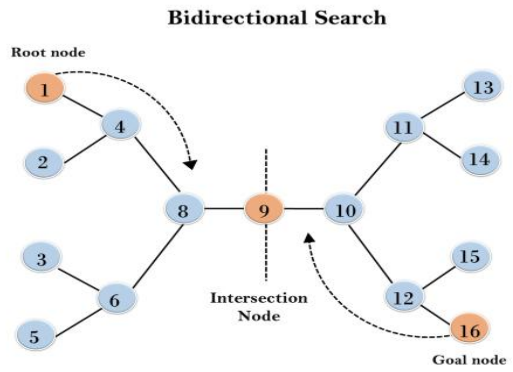
Bidirectional search algorithm runs two simultaneous searches, one form initial state called as forward-search and other from goal node called as backward-search, to find the goal node. Bidirectional search replaces one single search graph with two small sub graphs in which one starts the search from an initial vertex and other starts from goal vertex. The search stops when these two graphs intersect each other.

Bidirectional search can use search techniques such as BFS, DFS, DLS, etc.

Example:

In the below search tree, a bidirectional search algorithm is applied. This algorithm divides one graph/tree into two sub-graphs. It starts traversing from node 1 in the forward direction and starts from goal node 16 in the backward direction.

The algorithm terminates at node 9 where two searches meet.



Completeness: Bidirectional Search is complete if we use BFS in both searches.

Time Complexity: Time complexity of bidirectional search using BFS is **O(b^d)**.

Space Complexity: Space complexity of bidirectional search is **O(b^d)**.

Optimal: Bidirectional search is Optimal.

Lab Assignments

SET A

1. Write a Python Program to Implement Depth-First Search (DFS) Algorithm.
2. Write a Python Program to Implement Breadth First Search (BFS) Algorithm..

SET B

1. Write a Python Program to Implement A* algorithms.
2. Write a Python Program to Implement Depth Limited Search (DLS) Algorithm.

SET C

1. Write a Python Program to Implement Bidirectional Search Algorithm.

Assignment Evaluation

0: Not Done [ ]	1: Incomplete [ ]	2: Late Complete [ ]
3: Need Improvement [ ]	4: Complete [ ]	5: Well Done [ ]

Signature of Teacher

Assignment No 3
Introduction to Machine Learning

Objectives:
Students will be able to

- Learn about different types of machine learning, including supervised, unsupervised, and reinforcement learning.
- Study various machine learning algorithms and their applications in real-world problems.
- Gain familiarity with popular ML libraries such as Scikit-learn for implementing machine learning models.

Prerequisites:

- ✓ Basic statistics and probability knowledge.
- ✓ Programming skills in Python, especially with libraries like NumPy and Pandas.

Introduction:

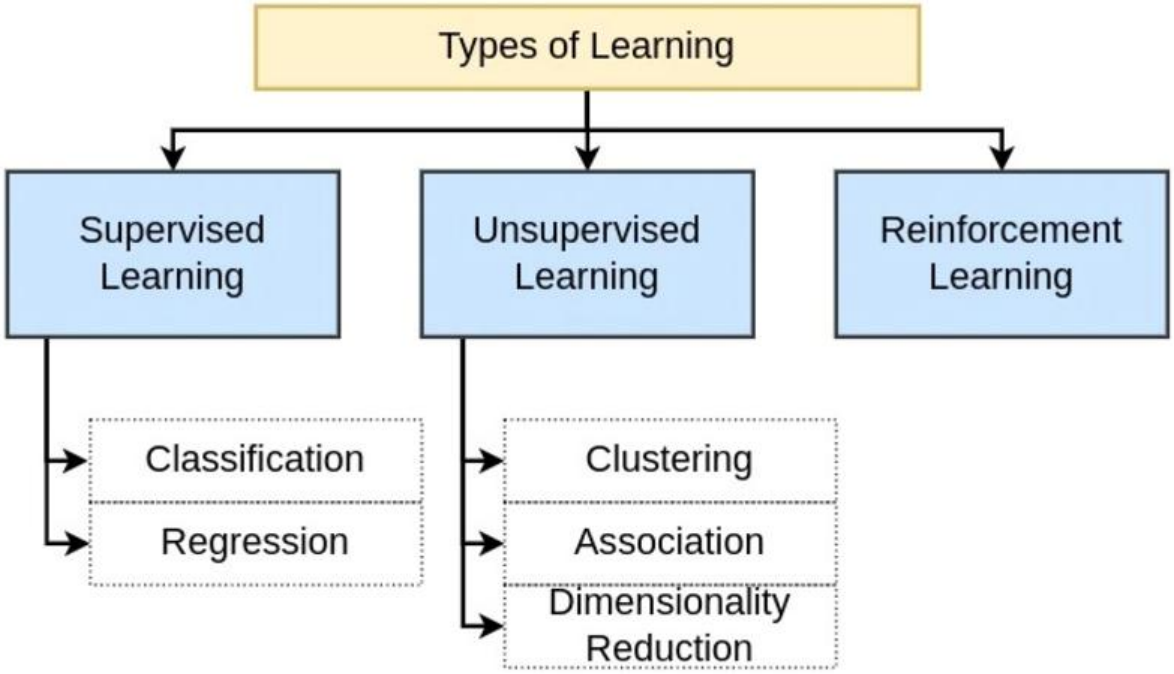
Machine learning (ML) is a branch of artificial intelligence (AI) that focuses on building systems that can learn and improve from experience without being explicitly programmed. Instead of relying on hardcoded rules, machine learning models use data to identify patterns, make decisions, or predict outcomes.

Key Concepts in Machine Learning:

1. Data: The foundation of machine learning, used to train models. Data can be structured (e.g., spreadsheets) or unstructured (e.g., images, audio).
2. Model: A mathematical representation of a process. Models are trained on data to make predictions or decisions.
3. Training: The process of feeding data to a model so it can learn patterns.
4. Features: Individual measurable properties or characteristics in the data.
5. Labels: The outcomes or target values in supervised learning.

Types of Machine Learning:

1. Supervised Learning: The model learns from labeled data.
    - Examples: Predicting house prices, classifying emails as spam or not.
    - Algorithms: Linear regression, decision trees, support vector machines



.

2. Unsupervised Learning: The model identifies patterns in unlabeled data.
   - Examples: Customer segmentation, anomaly detection.
   - Algorithms: K-means clustering, principal component analysis (PCA).
3. Reinforcement Learning: The model learns by interacting with an environment and receiving feedback in the form of rewards or penalties.
   - Examples: Robotics, game playing.
   - Algorithms: Q-learning, deep reinforcement learning.

Steps in Building a Machine Learning Model:

1. **Define the Problem**: Clearly understand the objective.
2. **Collect Data**: Gather relevant and high-quality data.
3. **Preprocess Data**: Clean and transform the data.
4. **Split Data**: Divide into training, validation, and test sets.
5. **Choose a Model**: Select an appropriate algorithm.
6. **Train the Model**: Feed the training data to the model.
7. **Evaluate the Model**: Test the model using the test data.
8. **Optimize and Deploy**: Improve performance and deploy the model.

Data preprocessing is a critical step in machine learning that involves preparing raw data for analysis and modeling. Proper preprocessing ensures that the data is clean, consistent, and suitable for feeding into machine learning algorithms, which can significantly improve model performance.

Tools for Preprocessing

- Python Libraries:
  - **Pandas**: Data manipulation and cleaning.
  - **NumPy**: Numerical operations.
  - **Scikit-learn**: Preprocessing utilities like `StandardScaler`, `LabelEncoder`, etc.

Steps to Load Dataset

1. Download the dataset and save it as `iris.csv`.
2. The dataset contains the following columns:
    - **SepalLength**: Length of the sepal (cm).
    - **SepalWidth**: Width of the sepal (cm).
    - **PetalLength**: Length of the petal (cm).
    - **PetalWidth**: Width of the petal (cm).
    - **Species**: Type of iris flower (setosa, versicolor, virginica).

Data Pre-processing

Data preprocessing is an essential step in data analysis and machine learning. It involves transforming raw data into a clean, organized, and suitable format for further analysis.

Steps involved in data preprocessing:

1. Data Collection:

- Collect the required data from various sources, such as databases, files, APIs, or web scraping.

2. Data Cleaning:

- Handle missing data: Identify missing values in the dataset and decide how to handle them. This may involve imputing missing values by filling them with appropriate values or removing rows or columns with missing data.

- Remove duplicates: Check for duplicate records in the dataset and eliminate them if necessary.

- Handle outliers: Detect and handle outliers that may significantly affect the analysis or modeling. This can be done by removing outliers, transforming them, or imputing them with appropriate values.

3. Data Integration:

- Merge datasets: If working with multiple datasets, merge them based on common key variables to create a unified dataset.

4. Data Transformation:

- Feature scaling: Scale numerical features to a similar range to prevent certain features from dominating the analysis. Common scaling techniques include normalization and standardization.

- Encoding categorical variables: Convert categorical variables into numerical form as many machine learning algorithms require numerical input. Techniques such as one-hot encoding or label encoding can be used depending on the nature of the data.

5. Data Reduction:

- Dimensionality reduction: If dealing with high-dimensional data, apply dimensionality reduction techniques like Principal Component Analysis (PCA) or Feature Selection to reduce the number of features while preserving the most relevant information.

6. Data Discretization:

- Convert continuous variables into categorical variables by dividing them into bins or intervals. This can simplify the analysis or allow for the application of certain algorithms that work better with categorical data.

7. Data Splitting:

- Split the preprocessed dataset into training and testing sets. The training set is used to train machine learning models, while the testing set is used to evaluate the performance of the trained models.

8. Data Normalization:

- Normalize the data to ensure that different features are on a similar scale. This helps prevent features with larger magnitudes from dominating the analysis or modeling process.

Data cleaning: this step involves identifying and removing missing, inconsistent, or irrelevant data. This can include removing duplicate records, filling in missing values, and handling outliers.

Handling Missing Values

Most of the datasets having contained missing values of NaN or null. We can handle these using the Python pandas module by either deleting them or replacing them with mean, mode, the most frequent value of the column.

1. find the missing values using isnull() function.

Example of finding missing values:

```
dataFrame.isnull( )
```

Example of finding missing values:

```
dataFrame.isnull( ).any( )
```

2. Removing Missing Values using dropna() function.

Example of dropping missing values:

```
dataFrame.dropna( )
```

3. Replacing with a value using fillna() function.

Example of replacing missing values with a constant value:

```
dataFrame.fillna(' ')
```

We can also use replace() function to replace with a constant value or with mean or other function.

Program to find all null values/missing values in a given dataset and remove them. Download dataset from github Play _tennis.csv

```
import pandas as pd

#reading the csv file

 df = pd.read_csv('D:\Dataset\Dataset/play_tennis.csv')

print("\nNull values in each column:")

print(df.isnull().sum()) #finding null values

df.dropna(axis=1) # Remove/drop null values

df.fillna('unknown') # fill missing values

print(df)
```

Data Transformation:

Converting categorical values into numerical format is essential to preprocess data for machine learning models. The choice between label encoding, one-hot encoding, or pandas get_dummies depends on the nature of the categorical variable and the requirements of the machine learning algorithm being used.

1. **Label Encoding** assigns a unique integer to each category in a categorical feature. For example, if a feature has categories like 'red', 'green', and 'blue', label encoding might assign them integer values like 0, 1, and 2 respectively.

Usage: Label encoding is suitable for ordinal categorical variables where the categories have a meaningful order.

2. **One-Hot Encoding** creates binary columns for each category in a categorical feature. Each category is represented as a binary vector (all zeros except for a one at the index corresponding to the category).

Usage: One-hot encoding is suitable for nominal categorical variables where there is no meaningful order among the categories.

3. **Panda**s **get_dummies** function is another way to perform one-hot encoding. It creates dummy variables for each category in a categorical feature and returns a DataFrame with these dummy variables.

Usage: Similar to one-hot encoding, get_dummies is useful for nominal categorical variables

Example:

```
from sklearn.preprocessing import LabelEncoder,OneHotEncoder

# Use LabelEncoder

le = LabelEncoder()

X['categorical_feature'] = le.fit_transform(X['categorical_feature'])

# Use OneHotEncoder

ohe=OneHotEncoder()

X_encoded = ohe.fit_transform(X[['categorical_feature']])

# Use pandas get_dummies function

X_encoded = pd.get_dummies(X['categorical_feature'])
```

Program to convert categorical values into numeric values using a given dataset.

```
import pandas as pd
from sklearn.preprocessing import LabelEncoder, StandardScaler, MinMaxScaler
# Reading the CSV file
df = pd.read_csv('play_tennis.csv')

# Displaying the dataset before conversion
print("Dataset before converting categorical to numeric:")
print(df)
# Convert categorical to numeric using LabelEncoder
le = LabelEncoder()
df['outlook'] = le.fit_transform(df['outlook'])
df['temp'] = le.fit_transform(df['temp'])
df['humidity'] = le.fit_transform(df['humidity'])
df['wind'] = le.fit_transform(df['wind'])
df['play'] = le.fit_transform(df['play'])
print("\nAfter converting categorical to numeric:")
print(df)
```

Feature Scaling:

Feature scaling in machine learning refers to the process of standardizing the range of independent variables or features of the data.

MinMaxScaler and **StandardScaler** are two popular techniques used for rescaling or standardizing features in machine learning and data preprocessing. Here's an explanation of each:

MinMaxScaler/(Normalization):

MinMaxScaler transforms features by scaling each feature to a given range. This range is often [0, 1] or [-1, 1].

The algorithm expects features to be on a specific scale (e.g., [0, 1] or [-1, 1]). You have a linear model or distance-based algorithms like K-Nearest Neighbors where feature magnitude matters.

StandardScaler/(Z-score Normalization):

StandardScaler standardizes features by removing the mean and scaling to unit variance.

The algorithm assumes normally distributed data or requires features to have a mean of 0 and a standard deviation of 1 (e.g., linear regression, logistic regression).

You have features with varying scales and you want to standardize them.

Write a python program to implement Data Normalization/Standardization - Rescale values for a given dataset.

```
import pandas as pd
from sklearn.preprocessing import LabelEncoder, StandardScaler, MinMaxScaler
df = pd.read_csv('play_tennis.csv')
le = LabelEncoder()
df['outlook'] = le.fit_transform(df['outlook'])
df['temp'] = le.fit_transform(df['temp'])
df['humidity'] = le.fit_transform(df['humidity'])
df['wind'] = le.fit_transform(df['wind'])
df['play'] = le.fit_transform(df['play'])

# Separate features (X) and target (y)
x = df.drop(columns=['play'])  # Independent variables or   x=df.iloc[:, :-1]
y = df['play']    # Dependent variable   or y = df.iloc[:, -1]
# Rescale values between 0 and 1 using MinMaxScaler
scaler = MinMaxScaler()
X_scaled = scaler.fit_transform(x)
# Displaying the scaled features
print("\nScaled features (MinMaxScaler):")
print(X_scaled)

# Alternatively, normalize data using StandardScaler
scaler_standard = StandardScaler()
X_standard_scaled = scaler_standard.fit_transform(x)

# Displaying the standardized features
print("\nStandardized features (StandardScaler):")
print(X_standard_scaled)
```

Train-Test Split:

The process of splitting a dataset into two subsets: one for training the model and the other for testing its performance, is commonly referred to as **train-test split** or **training and testing data splitting**.

- **Purpose**: Train-test split is used to evaluate the performance of a machine learning

model. The training set is used to train the model on known data, while the testing set is used to evaluate how well the model generalizes to new, unseen data.

- **Process**: Typically, the dataset is randomly partitioned into two subsets:
  - **Training Set**: This subset (~70-80% of the data) is used to train the model. The model learns the patterns in this data, adjusting its parameters to minimize prediction errors.
  - **Testing Set**: The remaining subset (~20-30% of the data) is used to evaluate the model's performance. It serves as a proxy for new data that the model hasn't seen before. By predicting outcomes on this set, we can assess how well the model generalizes to unseen data.

Example:

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

where,

X: Features (independent variables) of the dataset.

y: Target (dependent variable or labels) of the dataset.

test_size: Specifies the proportion of the dataset to include in the testing set (e.g., test_size=0.2 for 20% testing data).

random_state: Sets a seed for random number generation to ensure reproducibility.

Write a python program to split the dataset into training and test training dataset.

```
import pandas as pd
from sklearn.preprocessing import LabelEncoder, StandardScaler, MinMaxScaler
df = pd.read_csv('play_tennis.csv')
le = LabelEncoder()
df['outlook'] = le.fit_transform(df['outlook'])
df['temp'] = le.fit_transform(df['temp'])
df['humidity'] = le.fit_transform(df['humidity'])
df['wind'] = le.fit_transform(df['wind'])
df['play'] = le.fit_transform(df['play'])

# Separate features (X) and target (y)
x = df.drop(columns=['play'])  # Independent variables or   x=df.iloc[:, :-1]
y = df['play']    # Dependent variable   or y = df.iloc[:, -1]
# Rescale values between 0 and 1 using MinMaxScaler
scaler = MinMaxScaler()
X_scaled = scaler.fit_transform(x)
# Displaying the scaled features
print("\nScaled features (MinMaxScaler):")
print(X_scaled)

# Alternatively, normalize data using StandardScaler
scaler_standard = StandardScaler()
```

```
X_standard_scaled = scaler_standard.fit_transform(x)

# Displaying the standardized features
print("\nStandardized features (StandardScaler):")
print(X_standard_scaled)


from sklearn.model_selection import train_test_split

# Splitting the dataset into training and testing sets

x_train, x_test, y_train, y_test = train_test_split(x, y, test_size = 0.2, random_state = 50)

print("Training data:")

 print(x_train,y_train)

print("Testing data:")

 print(x_test,y_test)
```

Lab Assignments

SET A

1. Write a Python program to import necessary libraries to read a dataset and Describe the uses of Data Frame Attributes.(Use head(), tail(), shape, info() , describe() ,columns).

2. Write a Program to implement Data cleaning -find all null values/missing values in a given dataset and remove them. [use any dataset]

3. Write a python program to implement Reshaping/Transform data- Conversion of Categorical values in Numeric format for a given dataset.(LabelEncoder or One –Hot Encoder).[use any dataset]

SET B

1. Write a python program to implement Data Normalization/Standardization - Rescale values of columns using Standardscaler() or minmaxScaler() for a given dataset. [use any dataset]

2. Write a python program to implement Data Splitting: Split the given dataset into train set and test set which is of 70% & 30% respectively and print them.[use any dataset]

SET C

1. Write a python program to implement Pre-Processing of Play_Tennis.csv DataSet.

   - Import dataset & read a dataset.
   - Handle the Missing Values
   - Convert categorical data to numeric
   - Rescale/normalize the Data
   - Split the dataset into a train & test set and print them.

Assignment Evaluation

0: Not Done [ ]	1: Incomplete [ ]	2: Late Complete [ ]
3:Need Improvement []	4: Complete [ ]	5: Well Done [ ]

Signature of Teacher

Assignment No 4
Basics of Data and Statistics

Objectives:

Students will be able to

- Learn about different types of data (quantitative and qualitative) and their characteristics.
- Gain knowledge of descriptive and inferential statistics to analyze data effectively.
- Use Python libraries like Pandas for data manipulation and analysis.

Prerequisites:

- ✓ Basic knowledge of statistics.
- ✓ Familiarity with Python programming for data analysis tasks.

Introduction:

In data science, data refers to raw facts, statistics, or information that can be processed to extract insights. Data is the foundation for all data science processes and is used to build predictive models, uncover patterns, and derive actionable knowledge.

Types of Data in Data Science

1. **Based on Structure**
   - **Structured Data**: Organized in tables with rows and columns (e.g., customer database, financial transactions).
   - **Unstructured Data**: No predefined structure, requires tools like NLP or Computer Vision (e.g., text, images, videos).
   - **Semi-structured Data**: Partially organized, with markers (e.g., JSON, XML files, IoT sensor data).
2. **Based on Source**
   - **Primary Data**: Collected directly for a specific purpose (e.g., surveys, experiments).
   - **Secondary Data**: Collected by others for different purposes (e.g., census data, academic papers).
3. **Based on Nature of Values**
   - **Quantitative Data**: Numeric data.
     - **Discrete**: Countable (e.g., number of students).
     - **Continuous**: Measurable (e.g., temperature, weight).
   - **Qualitative Data**: Categorical data.
     - **Nominal**: No order (e.g., colors, gender).
     - **Ordinal**: Ordered (e.g., education levels).

4. **Based on Timeliness**
    - **Static Data**: Unchanged over time (e.g., historical records).
    - **Dynamic Data**: Continuously updated (e.g., live stock prices, GPS data).
5. **Based on Context**
    - **Big Data**: Large, complex datasets (e.g., social media data, e-commerce transactions).
    - **Metadata**: Data about data (e.g., photo file size, creation date).

Statistics is a branch of mathematics that deals with collecting, organizing, analyzing, interpreting, and presenting data. It helps in understanding and drawing meaningful insights from data, making it a cornerstone of data science.

Types of Statistics

1. **Descriptive Statistics**
    - Focuses on summarizing and describing the features of a dataset.
    - Methods include measures of central tendency (mean, median, mode), measures of dispersion (range, variance, standard deviation), and data visualization (charts, graphs).
2. **Inferential Statistics**
    - Makes predictions, inferences, or generalizations about a population based on a sample.
    - Methods include hypothesis testing, confidence intervals, and regression analysis.

Descriptive Statistics

1. **Calculate Measures of Central Tendency**
    - Find the mean, median, and mode of a dataset.
2. **Measure Dispersion**
    - Compute the variance, standard deviation, and range of a dataset.
3. **Visualize Data Distribution**
    - Create a histogram or box plot to visualize the distribution of data.
    - Generate a summary table for a dataset using pandas or numpy.
4. **Correlation Analysis**
    - Compute the correlation coefficient between two variables to understand their relationship.
5. **Frequency Distribution**
    - Create a frequency table or bar plot for categorical data.
6. **Identify Outliers**
    - Use the interquartile range (IQR) method to detect outliers in a dataset.

1. Write a Python Program to Calculate Measures of Central Tendency

```
import pandas as pd
from scipy import stats

# Load the dataset
data = pd.read_csv('iris.csv', header=None, names=['SepalLength', 'SepalWidth', 'PetalLength', 'PetalWidth', 'Species'])
```

```
# Calculate mean, median, and mode for SepalLength
mean = data['SepalLength'].mean()
median = data['SepalLength'].median()
mode = stats.mode(data['SepalLength'])

print(f"Mean: {mean}, Median: {median}, Mode: {mode.mode[0]}")
```

---

2. Write a Python Program to Calculate Measure Dispersion

```
import pandas as pd

# Load the dataset
data = pd.read_csv('iris.csv', header=None, names=['SepalLength', 'SepalWidth', 'PetalLength', 'PetalWidth', 'Species'])

# Variance, Standard Deviation, and Range for SepalWidth
variance = data['SepalWidth'].var()
std_dev = data['SepalWidth'].std()
data_range = data['SepalWidth'].max() - data['SepalWidth'].min()

print(f"Variance: {variance}, Standard Deviation: {std_dev}, Range: {data_range}")
```

---

3. Write a Python Program to Visualize Data Distribution.

```
import pandas as pd
import matplotlib.pyplot as plt

# Load the dataset
data = pd.read_csv('iris.csv', header=None, names=['SepalLength', 'SepalWidth', 'PetalLength', 'PetalWidth', 'Species'])

# Histogram
plt.hist(data['PetalLength'], bins=10, color='blue', alpha=0.7)
plt.title("Histogram of PetalLength")
plt.xlabel("Petal Length (cm)")
plt.ylabel("Frequency")
plt.show()

# Box Plot
plt.boxplot(data['PetalLength'])
plt.title("Box Plot of PetalLength")
plt.ylabel("Petal Length (cm)")
plt.show()
```

---

5. Write a Python Program to Perform Correlation Analysis.

```
import pandas as pd
```

```
# Load the dataset
data = pd.read_csv('iris.csv', header=None, names=['SepalLength', 'SepalWidth', 'PetalLength', 'PetalWidth', 'Species'])

# Correlation between SepalLength and PetalLength
correlation = data['SepalLength'].corr(data['PetalLength'])
print(f"Correlation Coefficient between SepalLength and PetalLength: {correlation}")
```

---

6. Write a Python Program to Create a Frequency Distribution

```
import pandas as pd
import matplotlib.pyplot as plt

# Load the dataset
data = pd.read_csv('iris.csv', header=None, names=['SepalLength', 'SepalWidth', 'PetalLength', 'PetalWidth', 'Species'])

# Frequency Table
frequency_table = data['Species'].value_counts()
print(frequency_table)

# Bar Plot
frequency_table.plot(kind='bar', color='skyblue')
plt.title("Frequency Distribution of Species")
plt.xlabel("Species")
plt.ylabel("Count")
plt.show()
```

---

7. Write a Python Program to Identify Outliers.

```
import pandas as pd

# Load the dataset
data = pd.read_csv('iris.csv', header=None, names=['SepalLength', 'SepalWidth', 'PetalLength', 'PetalWidth', 'Species'])

# IQR Method to Detect Outliers for SepalLength
Q1 = data['SepalLength'].quantile(0.25)
Q3 = data['SepalLength'].quantile(0.75)
IQR = Q3 - Q1

lower_bound = Q1 - 1.5 * IQR
upper_bound = Q3 + 1.5 * IQR

outliers = data[(data['SepalLength'] < lower_bound) | (data['SepalLength'] > upper_bound)]
print(f"Outliers in SepalLength:\n{outliers}")
```

Inferential Statistics

1. **Hypothesis Testing: P**erform a t-test (one-sample, two-sample, or paired) to test a hypothesis about a mean.
2. **ANOVA:** Use one-way ANOVA to compare means across multiple groups.
3. **Chi-Square Test:** Conduct a chi-square test of independence for categorical data.
4. **Confidence Intervals:** Calculate the confidence interval for a mean or proportion.

Lab Assignments

SET A

1. Write a Python Program to Import Data from CSV File Using Pandas and Calculate measures of central tendency.
2. Write a Python Program to Import Data from CSV File Using Pandas and Calculate measures of dispersion.

SET B

1. Write a Python Program to Implement Anova Test.
2. Write a Python Program to Perform t-Test.
3. Write a Python Program to Import Data from CSV File and Identify Outliers

SET C

1. Write a Python Program to Perform Correlation Analysis Between Numerical Features with the Help of Correlation Matrices Using Heatmaps with Seaborn.
2. Write a Python Program to Visualize Data/frequency Distributions using Histograms with Matplotlib or Seaborn.

Assignment Evaluation

0: Not Done [ ]	1: Incomplete [ ]	2: Late Complete [ ]
3: Need Improvement [ ]	4: Complete [ ]	5: Well Done [ ]

Signature of Teacher

Assignment No 5

Supervised Learning Model

Objectives:

Students will be able to

- Understand various supervised learning algorithms such as linear regression, logistic regression, decision trees, and support vector machines.
- Gain practical experience by implementing regression and classification models using Python.
- Learn how to assess the performance of machine learning models using appropriate metrics.

Prerequisites:

- ✓ Understanding of linear algebra concepts.
- ✓ Knowledge of Python programming for implementing ML algorithms.

Supervised Learning: Classification falls under supervised learning, where the algorithm learns from a labeled dataset that includes both input features (independent variables) and the desired output (dependent variable or class label).

Classification

Classification in data mining is a supervised learning technique used to predict the categorical class labels of new, unseen data based on past observations or labeled data.

Classification algorithms are used to forecast or classify the distinct values such as Real or False, Male or Female, Spam or Not Spam, etc.

Categorical Outcome: In classification, the goal is to predict the categorical class label of new observations or instances based on the patterns and relationships learned from the training data.

Classification Process:

- Training Phase: The algorithm learns from the labeled training dataset to build a model that can predict the class labels of new data points.
- Prediction Phase: Once trained, the model is used to classify or categorize new instances by applying the learned patterns and rules.

Performance Evaluation:

1. Accuracy: Measure of how often the classifier correctly predicts the class label.
2. Precision and Recall: Metrics to assess the classifier's performance on specific classes.
3. Confusion Matrix: Table used to evaluate the performance of a classification model.

Types of Classification Algorithms:

1. K-Nearest Neighbors (k-NN):

- **Type**: Instance-based learning
- **Description**: Classifies instances based on the majority class among the k-nearest neighbors. It is simple and effective for many types of data.

Algorithm Steps

1. **Input**:
    - Training set (Xtrain,ytrain)(X_{train}, y_{train})(Xtrain,ytrain)
    - Test instance xtestx_{test}xtest
    - Number of neighbors kkk
    - Distance metric d(x,y)d(x, y)d(x,y)
2. **Compute Distances**:
    - Calculate the distance between xtestx_{test}xtest and each instance xix_ixi in XtrainX_{train}Xtrain
3. **Sort Neighbors**:
    - Sort all training instances by their distance to xtestx_{test}xtest
4. **Select k Nearest Neighbors**:
    - Select the first kkk instances from the sorted list
5. **Vote for Class Labels**:
    - Determine the majority class among the kkk nearest neighbors
6. **Output**:
    - The predicted class label for xtestx_{test}xtest

Write a python program to perform k-nearest neighbor classification using in-built iris dataset,

```
# Importing necessary libraries
from sklearn.neighbors import KNeighborsClassifier

# Load the Iris dataset
iris = datasets.load_iris()
X = iris.data  # Features
y = iris.target  # Target variable

# Splitting the dataset into training and testing sets (if not split already)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Creating KNN classifier
knn_classifier = KNeighborsClassifier(n_neighbors=5)  # You can adjust the number of neighbors as needed

# Training the classifier
knn_classifier.fit(X_train, y_train)

# Making predictions on the testing set
```

```
y_pred = knn_classifier.predict(X_test)

# Calculating the accuracy
accuracy = accuracy_score(y_test, y_pred)
print("Accuracy:", accuracy)
```

2. Support Vector Machines (SVM):

- **Type**: Linear and non-linear model
- **Description**: Find the optimal hyperplane that best separates classes in the feature space. It uses kernel tricks to handle non-linearly separable data.
- **Hyperplane**: SVM aims to find the optimal hyperplane that best separates the classes in the feature space.
- **Margin**: The distance between the hyperplane and the nearest data points from either class, called support vectors. SVM maximizes this margin.
- **Support Vectors**: The data points that are closest to the hyperplane and influence its position and orientation.
- **Kernel Trick**: SVM can transform data into higher dimensions using kernel functions (e.g., linear, polynomial, radial basis function (RBF)) to handle non-linear relationships.

Algorithm Steps:

1. **Input**: Training data (Xtrain,ytrain)(X_{train}, y_{train})(Xtrain,ytrain), test instance xtestx_{test}xtest, kernel function KKK, regularization parameter CCC.
2. **Training**:
   - Solve the optimization problem to find the hyperplane that maximizes the margin: Identify support vectors.
3. **Prediction**:
   - Compute the decision function for a test instance:
   - Predict the class label based on the sign of f(xtest)f(x_{test})f(xtest).

Write a python program to perform support vector machine classification using in-built iris dataset.

```
# Importing necessary libraries
import numpy as np
from sklearn import datasets
from sklearn.model_selection import train_test_split
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score

# Load the Iris dataset
iris = datasets.load_iris()
X = iris.data  # Features
y = iris.target  # Target variable

# Splitting the dataset into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

```
# Creating SVM classifier
svm_classifier = SVC(kernel='linear')  # Linear kernel
# Other options for the kernel parameter are 'poly', 'rbf', 'sigmoid', etc.

# Training the classifier
svm_classifier.fit(X_train, y_train)

# Making predictions on the testing set
y_pred = svm_classifier.predict(X_test)

# Calculating the accuracy
accuracy = accuracy_score(y_test, y_pred)
print("Accuracy:", accuracy)
```

3. Decision Trees:

- **Type**: Build a tree-like model of decisions based on the data features.
- **Description**: Splits the data into subsets based on the value of input features. Each node represents a feature, each branch represents a decision rule, and each leaf node represents an outcome.

Algorithm Steps

1. **Input**:
   - Training data (Xtrain,ytrain)(X_{train}, y_{train})(Xtrain,ytrain)
   - Test instance xtestx_{test}xtest
2. **Build the Tree**:
   - **Start at Root**: Use the entire dataset.
   - **Feature Selection**: Choose the best feature to split data (e.g., using Gini impurity, Information Gain).
   - **Split Data**: Divide dataset based on the selected feature.
   - **Recursion**: Repeat for each child node until stopping criteria (e.g., max depth, min samples) are met.
3. **Prediction**:
   - **Traverse the Tree**: Follow the path according to the feature values of xtestx_{test}xtest from root to leaf.
   - **Leaf Node**: The output is the class label (classification) or mean value (regression) of the leaf.

Feature Selection: Choose the best feature to split data.

Stopping Criteria: Prevent overfitting with max depth, min samples, etc.

Prediction: Traverse from root to leaf for the final prediction.

Write a python program to perform Decision Tree classification using in-built iris dataset.

```
# Importing necessary libraries
from sklearn.tree import DecisionTreeClassifier
```

```
# Load the Iris dataset
iris = datasets.load_iris()
X = iris.data  # Features
y = iris.target  # Target variable

# Splitting the dataset into training and testing sets (if not split already)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Creating Decision Tree classifier
decision_tree_classifier = DecisionTreeClassifier()

# Training the classifier
decision_tree_classifier.fit(X_train, y_train)

# Making predictions on the testing set
y_pred = decision_tree_classifier.predict(X_test)

# Calculating the accuracy
accuracy = accuracy_score(y_test, y_pred)
print("Accuracy:", accuracy)
```

4. Naive Bayes:

- **Type**: Probabilistic model
- **Description**: Based on Bayes' theorem, it assumes independence between features. It is particularly effective for text classification tasks like spam detection.

Algorithm Steps

1. **Input**:
   - Training set (Xtrain,ytrain)(X_{train}, y_{train})(Xtrain,ytrain)
   - Test instance xtestx_{test}xtest
2. **Training**:
   - **Calculate Prior Probabilities**:
   - **Calculate Likelihood**:Use Gaussian distribution for continuous features.
3. **Prediction**:
   - **Calculate Posterior Probabilities**:
   - **Choose Class with Highest Posterior Probability**

Write a python program to perform Naive Bayes classification using in-built iris dataset,.

```
# Importing necessary libraries
from sklearn.naive_bayes import GaussianNB

# Load the Iris dataset
iris = datasets.load_iris()
X = iris.data  # Features
y = iris.target  # Target variable

# Splitting the dataset into training and testing sets (if not split already)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

```
# Creating Naive Bayes classifier
naive_bayes_classifier = GaussianNB()

# Training the classifier
naive_bayes_classifier.fit(X_train, y_train)

# Making predictions on the testing set
y_pred = naive_bayes_classifier.predict(X_test)

# Calculating the accuracy
accuracy = accuracy_score(y_test, y_pred)
print("Accuracy:", accuracy)
```

Regression

It is a data Mining Technique that is used for predicting a range of continuous values in a specific dataset.
It analyses the relationship between a target variable (dependent) and its predictor variable (independent)
Regression involves the technique of fitting a straight line or a curve on numerous data points. It happens in such a way that the distance between the data points and cure comes out to be the lowest.
Regression is used across multiple industries for business and marketing planning, financial forecasting, environmental modelling and analysis of trends.

Types of Regression -

- **Linear Regression :**
    - **Simple Linear Regression:**
    - **Multiple linear Regression**
- **Non -linear Regression**
    - **Polynomial Regression:**

Linear regression is one of the easiest and most popular Machine Learning algorithms. It is a statistical method that is used for predictive analysis. Linear regression makes predictions for continuous/real or numeric variables such as **sales, salary, age, product price,** etc.

Linear regression algorithm shows a linear relationship between a dependent (y) and one or more independent (y) variables, hence called linear regression. Since linear regression shows the linear relationship, which means it finds how the value of the dependent variable is changing according to the value of the independent variable.

The linear regression model provides a sloped straight line representing the relationship between the variables.

Mathematically, we can represent a linear regression as:
$Y = a + bX$
Where,

a represents the intercept.
b represents the slope of the regression line.
X and Y represent the predictor and target variables, respectively.

Types of Linear regression:

Simple Linear Regression:

If a single independent variable is used to predict the value of a numerical dependent variable, then such a Linear Regression algorithm is called Simple Linear Regression.

Mathematically, a linear regression can be represented as:

$$Y = a + bX$$

Write a Program to implement simple linear regression using Sample Data - Salary and Years of Experience

```
import pandas as pd

from sklearn.linear_model import LinearRegression

from sklearn.model_selection import train_test_split

from sklearn.metrics import mean_squared_error

from sklearn.preprocessing import StandardScaler

# Create the sample data as a dictionary

data = {

    'Years_of_Experience': [1, 2, 3, 4, 5, 6, 7, 8, 9, 10],

    'Salary': [45000, 48000, 51000, 54000, 57000, 60000, 63000, 66000, 69000, 72000]

}

# Convert the dictionary to a pandas DataFrame

df = pd.DataFrame(data)

# Show the DataFrame

print("Sample Data:")

print(df)

# Features (independent variable): Years_of_Experience

X = df[['Years_of_Experience']]  # Independent variable

# Target variable (dependent variable): Salary
```

```
y = df['Salary']

# Split the dataset into training and testing sets

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Initialize and train the Simple Linear Regression model

model = LinearRegression()

model.fit(X_train, y_train)

# Predict the salary using the test set

y_pred = model.predict(X_test)

# Evaluate the model using Mean Squared Error (MSE)

mse = mean_squared_error(y_test, y_pred)

print(f'\nMean Squared Error: {mse:.2f}')

# Print the coefficient (slope) and intercept of the regression model

print(f'\nCoefficient (Slope): {model.coef_[0]}')

print(f' Intercept: {model.intercept_}')

# Predict salary for a new employee (example: Years_of_Experience=6)

new_data = [[6]]  # New employee with 6 years of experience

predicted_salary = model.predict(new_data)

print(f'\nPredicted salary for the new employee with 6 years of experience: {predicted_salary[0]:.2f}')
```

Multiple Linear regression:

If more than one independent variable is used to predict the value of a numerical dependent variable, then such a Linear Regression algorithm is called Multiple Linear Regression.

Mathematically, Multiple regression can be represented as:
Y = a + b1X1+ b2X2 + b3X3 + ... + btXt

Write a Program to implement multiple linear regression using Sample Data - Salary and Years of Experience,Age and Performance of employee.

```
import pandas as pd
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split
```

```
from sklearn.metrics import mean_squared_error
from sklearn.preprocessing import StandardScaler

# Create the sample data as a dictionary
data = {
    'Years_of_Experience': [1, 2, 3, 4, 5, 6, 7, 8, 9, 10],
    'Age': [22, 23, 25, 26, 28, 30, 32, 34, 36, 40],
    'Performance': [50, 55, 60, 65, 70, 75, 80, 85, 90, 95],
    'Salary': [45000, 48000, 51000, 54000, 57000, 60000, 63000, 66000, 69000, 72000]
}

# Convert the dictionary to a pandas DataFrame
df = pd.DataFrame(data)

# Show the DataFrame
print(df)

# Features (independent variables): Years_of_Experience, Age, Performance
X = df[['Years_of_Experience', 'Age', 'Performance']]  # Independent variables

# Target variable (dependent variable): Salary
y = df['Salary']

# Split the dataset into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Standardize the features
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# Initialize and train the Multiple Linear Regression model
model = LinearRegression()
model.fit(X_train, y_train)

# Predict the salary using the test set
y_pred = model.predict(X_test)

# Evaluate the model using Mean Squared Error (MSE)
mse = mean_squared_error(y_test, y_pred)
print(f'Mean Squared Error: {mse:.2f}')

# Print the coefficients and the intercept of the regression model
print(f'Coefficients: {model.coef_}')
print(f'Intercept: {model.intercept_}')

# Predict salary for a new employee (example: Years_of_Experience=5, Age=29, Performance=72)
new_data = [[5, 29, 72]]
new_data_scaled = scaler.transform(new_data)
predicted_salary = model.predict(new_data_scaled)
print(f'Predicted salary for the new employee: {predicted_salary[0]:.2f}')
```

Non-Linear regression:

Nonlinear regression refers to a broader category of regression models where the relationship between the dependent variable and the independent variables is not assumed to be linear.

If the underlying pattern in the data exhibits a curve, whether it's exponential growth, decay, logarithmic, or any other non-linear form, fitting a nonlinear regression model can provide a more accurate representation of the relationship. This is because in linear regression it is pre-assumed that the data is linear.

A simple nonlinear regression model is expressed as follows:

$Y = f(X,\beta) + \epsilon$

Where:

- X is a vector of P predictors
- β is a vector of k parameters
- F (-) is the known regression function
- ϵ is the error term

Polynomial Regression is an extension of Linear Regression where the predictor variable xxx is raised to a power to capture non-linear relationships between the independent variable and the dependent variable.

For a polynomial regression of degree n, the equation is:

$$y=\beta 0+\beta 1x+\beta 2x2+\beta 3x3+\cdots+\beta nxn+\epsilon$$

Where:

- y is the dependent variable (the target you're trying to predict),
- x is the independent variable (the feature used for prediction),
- β0 is the intercept (constant term),
- β1,β2,…,βn are the coefficients (parameters) for each term of x.
- x1,x2,…,xn represent the polynomial terms (degree n),
- ϵ is the error term (residuals, or the difference between the actual and predicted values).

Python program to implement Polynomial Regression (non-linear regression) using sample data such as Salary, Years of Experience, Age, and Performance of employees.

```
import numpy as np
import pandas as pd
from sklearn.preprocessing import PolynomialFeatures
from sklearn.linear_model import LinearRegression
import matplotlib.pyplot as plt
from sklearn.metrics import mean_squared_error
# Sample Data: Years of Experience, Age, Performance, and Salary
data = {
    'Years_of_Experience': [1, 2, 3, 4, 5, 6, 7, 8, 9, 10],
    'Age': [22, 25, 27, 30, 32, 35, 38, 40, 43, 45],
    'Performance': [55, 60, 65, 70, 75, 80, 85, 90, 92, 95],
```

```
    'Salary': [45000, 48000, 51000, 54000, 57000, 60000, 63000, 66000, 69000, 72000]
}
# Convert the dictionary to a pandas DataFrame
df = pd.DataFrame(data)
# Display the dataset
print("Dataset:\n", df)
# Independent variable (Features): Years_of_Experience, Age, and Performance
X = df[['Years_of_Experience', 'Age', 'Performance']]  # Features
# Dependent variable (Target): Salary
y = df['Salary']  # Target variable
# Polynomial Feature Transformation (Degree 2)
poly = PolynomialFeatures(degree=2)
X_poly = poly.fit_transform(X)  # Transform the features into polynomial features
# Initialize and train the Polynomial Regression model
model = LinearRegression()
model.fit(X_poly, y)
# Predict Salary using the model
y_pred = model.predict(X_poly)
# Evaluate the model with Mean Squared Error (MSE)
mse = mean_squared_error(y, y_pred)
print(f'Mean Squared Error: {mse:.2f}')
# Print model coefficients and intercept
print(f'Coefficients: {model.coef_}')
print(f'Intercept: {model.intercept_}')

# Predict Salary for a new employee
new_data = np.array([[6, 32, 80]])  # Example: 6 years of experience, 32 years old, performance 80
new_data_poly = poly.transform(new_data)  # Transform the new data using PolynomialFeatures
predicted_salary = model.predict(new_data_poly)
print(f'Predicted Salary for the new employee: ${predicted_salary[0]:.2f}')

# Plotting the Polynomial Regression curve (for Years_of_Experience vs Salary)
plt.scatter(df['Years_of_Experience'], df['Salary'], color='blue')
plt.plot(df['Years_of_Experience'],    model.predict(poly.transform(df[['Years_of_Experience',    'Age',
'Performance']])), color='red')
plt.title('Polynomial Regression: Salary vs Years of Experience')
plt.xlabel('Years of Experience')
plt.ylabel('Salary')
plt.show()
```

Lab Assignments

SET A

1. Write a Python program to apply **simple linear regression** and predict the **score** of a student if he/she has studied for **9 hours**. The dataset includes the study hours and the corresponding scores of students.
   (Data set link: https://drive.google.com/open?id=1oakZCv7g3mlmCSdv9J8kdSaqO5_6dIOw)
   data = { 'Hours': [2.5, 5.1, 3.2, 8.5, 3.5, 1.5, 9.2, 5.5, 8.3, 2.7, 7.7, 5.9, 4.5, 3.3, 1.1, 8.9, 2.5, 1.9, 6.1, 7.4, 2.7, 4.8], 'Scores': [21, 47, 27, 75, 30, 20, 88, 60, 81, 25, 85, 62, 41, 42, 17, 95, 30, 24, 67, 69, 30, 54] }

2. Write a Python program to apply **polynomial regression** to predict the car price based on engine size, horsepower, and curb weight.

   (Dataset Link: https://www.kaggle.com/datasets/hellbuoy/car-price-prediction/)

   data = { 'Engine Size': [130, 150, 200, 120, 180, 140, 250, 300, 230, 160], 'Horsepower': [110, 130, 150, 95, 160, 120, 190, 210, 180, 140], 'Curb Weight': [2500, 2700, 3000, 2400, 3200, 2600, 3500, 4000, 3300, 2800], 'Price': [15000, 18000, 25000, 14000, 22000, 16000, 28000, 35000, 27000, 19000] }

3. Write a Python program to apply **multiple regression** to predict **housing prices** based on features such as **area**, **bedrooms**, and **age**. The dataset is stored in a CSV file, and you will use pandas to load the data and scikit-learn for the regression analysis.Predict housing price for a new data point (example: area=2000, bedrooms=3, age=10).

   (Dataset Link: https://www.kaggle.com/datasets/vikasukani/mini-house-price-data-set/)

   data = { 'area': [1500, 1800, 2400, 3000, 3500, 4000, 2200, 2500, 3500, 4200], 'bedrooms': [3, 4, 4, 5, 5, 6, 3, 4, 4, 6], 'age': [10, 15, 20, 5, 10, 8, 12, 7, 18, 6], 'price': [400000, 450000, 500000, 700000, 750000, 850000, 475000, 600000, 650000, 950000] }

SET B

1. Write a python program to Build Decision Tree Classification model using Play Tennis dataset to predict [0:overcast,2:Mild,1:normal ,0:Weak] tuple belongs to which class whether play sports or not.
2. Write a python program to implement Naive Bayes Classifier Model using Play Tennis to predict [0:overcast,2:Mild,1:normal ,0:Weak] tuple belongs to which class whether play sports or not.
3. Write a python program to implement the KNeighborsClassifier model and check model accuracy and confusion matrix. use a dataset is available in the sklearn library (from sklearn.datasets import load_wine) or from sklearn.datasets import load_diabetes)

SET C

1. Write a python program to build Support Vector Machine Model for breast cancer dataset and check accuracy of model with precision and recall.The dataset is available in the sklearn library .(from sklearn.datasets import load_breast_cancer)
2. Case Study: Every year, students take the GRE to secure admissions to foreign universities. The dataset contains the following attributes: GRE Score (out of 340), TOEFL Score (out of 120), University Rating (out of 5), SOP strength (out of 5), LOR strength (out of 5), UG GPA (out of 10), Research Experience (0=no, 1=yes), and Admission Status (0=no, 1=yes). The goal is to predict Admission Status based on GRE and Academic Scores.

   Task:

   1. Dataset: Download the dataset from Kaggle.
   2. Preprocessing: Apply techniques like Label Encoding and Data Transformation, if necessary.
   3. Data Preparation: Split the dataset into training and testing sets.
   4. Model Building: Use a random forest/logistic regression Classifier to predict admission status.
   5. Evaluation: Evaluate the model using appropriate metrics and suggest improvements.

Assignment Evaluation

0: Not Done [ ]	1: Incomplete [ ]	2: Late Complete [ ]
3:Need Improvement [ ]	4: Complete [ ]	5: Well Done [ ]

Signature of Teacher

Assignment No 6
Unsupervised Learning Model

Objectives:

Students will be able to

- Learn about clustering methods (e.g., K-Means, hierarchical clustering) and association analysis (e.g., Apriori algorithm).
- Gain hands-on experience by applying clustering algorithms to real datasets using Python.
- Understand how to identify patterns in data without labeled responses through unsupervised learning techniques.

Prerequisites:

✓ Basic knowledge of statistics and probability.
✓ Familiarity with Python programming for implementing unsupervised learning algorithms.

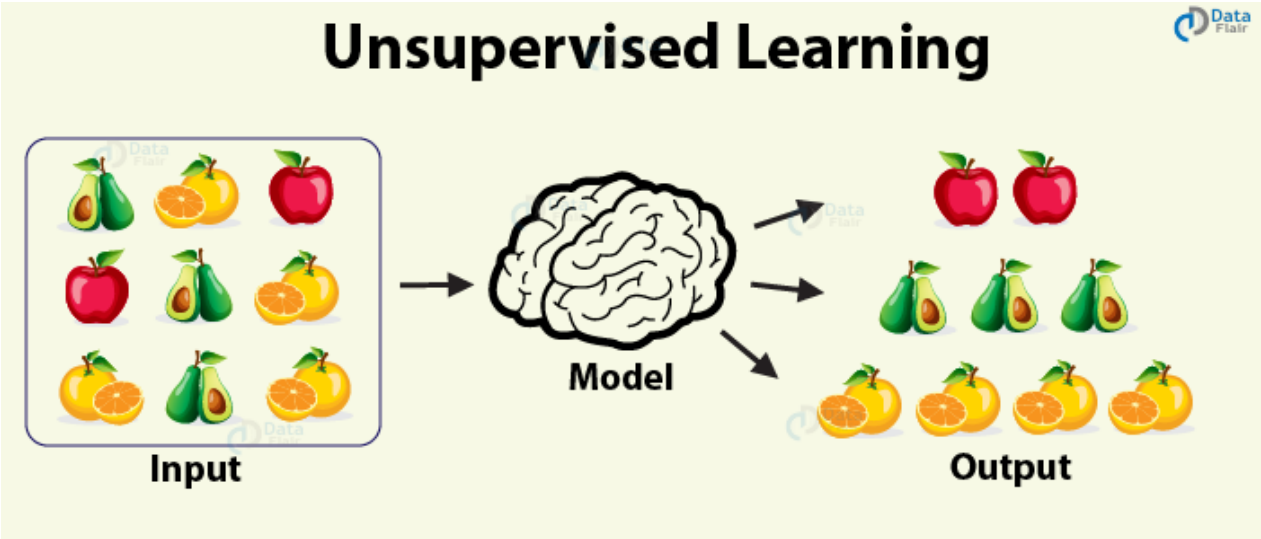
Introduction:

Unsupervised Learnin**g** algorithm, the data is not explicitly labeled into different classes, that is, there are no labels. The model is able to learn from the data by finding implicit patterns.

Unsupervised Learning algorithms identify the data based on their densities, structures, similar segments, and other similar features. Unsupervised Learning Algorithms are based on Hebbian Learning.

Cluster analysis is one of the most widely used techniques in supervised learning.

Let us look at some of the important algorithms that come under Unsupervised Learning.



Clustering

Clustering, also known as cluster analysis, is a technique of grouping similar sets of objects in the same

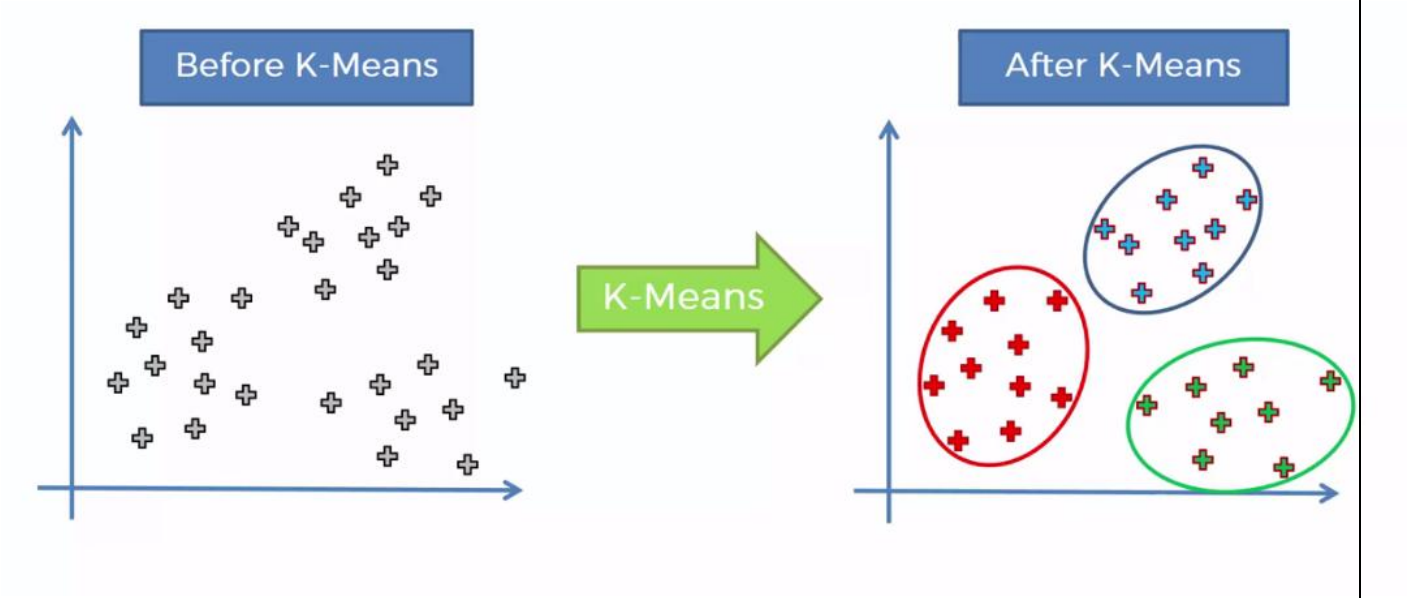
group that are different from the objects in another group.

Clustering is a set of techniques used to partition data into groups, or clusters.

Some of the essential clustering techniques are as follows –

a. K-means

The aim of the k-means clustering algorithm is to partition the n observations in the data into k clusters such that each observation belongs to the cluster with the nearest mean.k-means clustering method is an unsupervised machine learning technique used to identify clusters of data objects in a dataset.



This serves as the prototype of the cluster.

b. Hierarchical clustering

Hierarchical clustering determines cluster assignments by building a hierarchy. This is implemented by either a bottom-up or a top-down approach:

- Agglomerative clustering is the bottom-up approach. It merges the two points that are the most similar until all points have been merged into a single cluster.
- Divisive clustering is the top-down approach. It starts with all points as one cluster and splits the least similar clusters at each step until only single data points remain.

These methods produce a tree-based hierarchy of points called a dendrogram.

K-Means Clustering Algorithm Steps:

1. **Initialize Centroids**:
   Start by randomly selecting kkk centroids from the data.

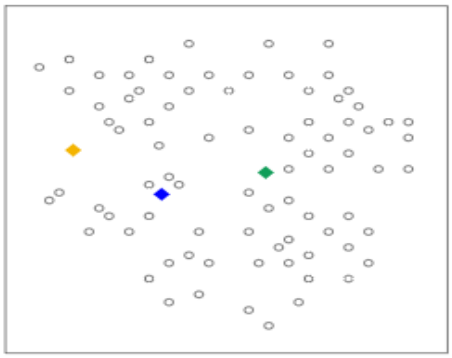


Figure 1: k-means at initialization.

2. **Assign Points to Nearest Centroid**:
   Each data point is assigned to the nearest centroid, forming kkk clusters.

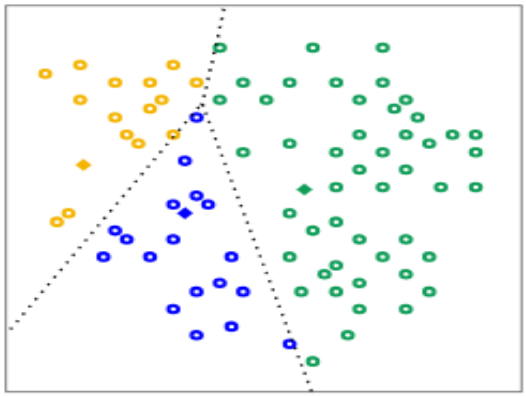


Figure 2: Initial clusters.

3. **Recalculate Centroids**:
   For each cluster, compute the new centroid as the mean of all points in that cluster.

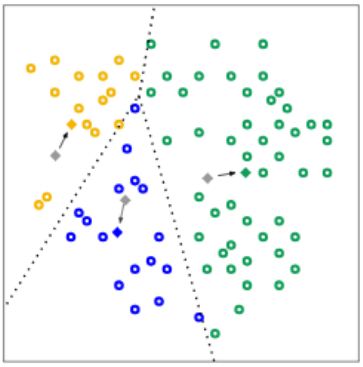


Figure 3: Recomputed centroids.

4. **Reassign Points to New Centroids**:
   Points are reassigned to the nearest centroid based on the updated positions.

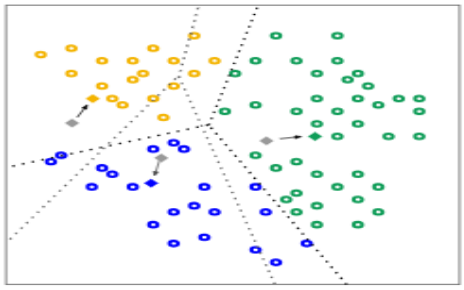


Figure 4: Clusters after reassignment.

5. **Repeat**:
   Steps 3 and 4 are repeated until the centroids no longer change or a stopping criterion is met (e.g., max iterations).

Write a python program to demonstrate k means clustering using mallcustomers.csv dataset.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans
from sklearn.metrics import confusion_matrix

# Load the dataset
data = pd.read_csv('Mall_Customers.csv')

# Use the relevant features
X = data.iloc[:, [3, 4]].values  # Annual Income and Spending Score

# K-Means Clustering
kmeans = KMeans(n_clusters=3, init='k-means++', random_state=42)
kmeans_labels = kmeans.fit_predict(X)


# Visualization
plt.figure(figsize=(14, 6))

# K-Means Clustering Visualization
plt.subplot(1, 2, 1)
plt.scatter(X[:, 0], X[:, 1], c=kmeans_labels, cmap='viridis', s=50)
plt.scatter(kmeans.cluster_centers_[:, 0], kmeans.cluster_centers_[:, 1], s=300, c='red',
label='Centroids')
plt.title('K-Means Clustering')
plt.xlabel('Annual Income')
plt.ylabel('Spending Score')
plt.legend()

print("K-Means Clustering Labels:", kmeans_labels[:10])
```

Write a python program to demonstrate hierarchical clustering using mallcustomers.csv dataset.

```
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd
from sklearn.cluster import AgglomerativeClustering
from scipy.cluster.hierarchy import dendrogram, linkage

# Load the dataset
data = pd.read_csv('Mall_Customers.csv')

# Use the relevant features (Annual Income and Spending Score)
X = data.iloc[:, [3, 4]].values

# Perform Hierarchical Clustering
hierarchical = AgglomerativeClustering(n_clusters=3, affinity='euclidean', linkage='ward')
hierarchical_labels = hierarchical.fit_predict(X)

# Visualization of Hierarchical Clustering

# Dendrogram Plot to visualize the hierarchical relationships
plt.figure(figsize=(14, 6))

# Dendrogram
plt.subplot(1, 2, 1)
linked = linkage(X, method='ward')  # Ward's method for linkage
dendrogram(linked)
plt.title('Dendrogram')
plt.xlabel('Customers')
plt.ylabel('Euclidean Distance')

# Hierarchical Clustering Visualization
plt.subplot(1, 2, 2)
plt.scatter(X[:, 0], X[:, 1], c=hierarchical_labels, cmap='viridis', s=50)
plt.title('Hierarchical Clustering')
plt.xlabel('Annual Income')
plt.ylabel('Spending Score')
plt.legend()

# Show the plots
plt.tight_layout()
plt.show()

print("Hierarchical Clustering Labels:", hierarchical_labels[:10])
```

Association discovers the association or connection among a set of items. Association identifies the relationships between objects. Association analysis is used for commodity management, advertising, catalog design, direct marketing etc. A retailer can identify the products that normally customers purchase together or even find the customers who respond to the promotion of the same kind of products. If a retailer finds that beer and nappy are bought together mostly, he can put

nappies on sale to promote the sale of beer.
Frequent patterns are patterns (e.g., itemsets, subsequences, or substructures) that appear frequently in a data set. For example, a set of items, such as milk and bread, that appear frequently together in a transaction data set is a frequent itemset.
A typical example of frequent itemset mining is market basket analysis. This process analyzes customer buying habits by finding associations between the different items that customers place in their "shopping baskets"

In general, association rule mining can be viewed as a two-step process:
1. Find all frequent itemsets: By definition, each of these itemsets will occur at least as frequently as a predetermined minimum support count, min_sup.
2. Generate strong association rules from the frequent itemsets: By definition, these rules must satisfy minimum support and minimum confidence. the second step is much less costly than the first, the overall performance of mining association rules is determined by the first step.

What is the Apriori Algorithm?

The **Apriori algorithm** is a method used to identify **frequent itemsets** in a dataset and generate **association rules**. It works specifically with Boolean (binary) association rules and relies on **prior knowledge** about itemset properties.

1. **Apriori Property**:
   If an itemset is frequent, then all its subsets must also be frequent. This property helps to **prune the search space** by eliminating itemsets whose subsets are infrequent.
2. **Level-Wise Search**:
   The algorithm starts with **1-itemsets** (individual items) and iteratively builds larger itemsets:
   - From 1-itemsets to 2-itemsets, then 3-itemsets, and so on.
   - For each iteration, the database is scanned to count itemset occurrences and verify support.

How Apriori Works (Step-by-Step):

1. **Find Frequent 1-Itemsets (L1)**:
   Scan the database to count the occurrences of each item and keep those that meet the minimum support threshold. This gives L1
2. **Generate Candidate k-Itemsets (Ck)**:
   `Combine the frequent (k−1) itemsets to create` **candidate k-itemsets**.
3. **Prune Candidates**:
   Use the Apriori property to remove candidate itemsets where any subset is not frequent.
4. **Count and Verify**:
   Scan the database to count the support of the remaining candidates. Keep only the itemsets meeting the minimum support to form Lk .
5. **Repeat**:
   Continue this process until no more frequent itemsets can be generated.

Advantages of Apriori:

- Reduces computation by pruning the search space with the Apriori property.

- Easy to understand and implement.

Limitations:

- Requires multiple database scans, which can be costly for large datasets.
- May not be efficient for dense datasets with many frequent itemsets.

Frequent Pattern Growth (FP-Growth)

The **FP-Growth algorithm** is an improvement over the Apriori algorithm for mining frequent itemsets without generating candidates.

Key Features:

1. **FP-Tree**:
   The database is compressed into a **Frequent Pattern Tree (FP-tree)** that retains itemset associations, reducing redundancy.
2. **Divide-and-Conquer Strategy**:
   - The database is divided into **conditional databases** based on frequent items (called "pattern fragments").
   - Each pattern fragment is mined separately.

Advantages:

- No candidate generation.
- More efficient than Apriori, especially for large datasets with dense itemsets.

- `mlxtend`: For implementing the Apriori algorithm and association rule generation

Install Required Libraries:
If you haven't installed `mlxtend`, run the following command:
bash
Copy code
```
pip install mlxtend
```

Write a python program to generate frequent itemsets and association rules using Apriori Algorithm.

Tasks:

1. Preprocess the data (note: dataset has no header).
2. Generate a list of transactions.
3. Train the Apriori algorithm.
4. Visualize the generated rules.
5. Adjust hyperparameters (e.g., minimum confidence) and observe changes in the rules.

```
import pandas as pd

from mlxtend.preprocessing import TransactionEncoder
```

```
from mlxtend.frequent_patterns import apriori, association_rules

# Step 1: Load the Dataset

# Here, the dataset doesn't have headers, so we'll manually load and preprocess it.

data = []

with open('Market_Basket_Optimisation.csv', 'r') as file:

    for line in file:

        data.append(line.strip().split(','))

# Step 2: Preprocess the Data to create a list of transactions

print("Sample Transactions:")

print(data[:5])  # Displaying the first 5 transactions

# Step 3: Apply the TransactionEncoder to convert the data into a format suitable for the apriori algorithm

te = TransactionEncoder()

encoded_data = te.fit_transform(data)

df = pd.DataFrame(encoded_data, columns=te.columns_)

# Step 4: Apply Apriori algorithm to find frequent itemsets

min_support = 0.01  # You can change this threshold based on your data

freq_itemsets = apriori(df, min_support=min_support, use_colnames=True)

# Display frequent itemsets

print("\nFrequent Itemsets:")

print(freq_itemsets)

# Step 5: Generate association rules using the frequent itemsets

min_confidence = 0.2  # Minimum confidence threshold

rules = association_rules(freq_itemsets, metric="confidence", min_threshold=min_confidence)


# Display the generated rules

print("\nGenerated Association Rules:")

print(rules[['antecedents', 'consequents', 'support', 'confidence', 'lift']])
```

```
# Step 6: Visualize or inspect the top rules

print("\nTop Rules (support, confidence, lift):")

print(rules[['antecedents', 'consequents', 'support', 'confidence', 'lift']].head())

# Adjust Hyperparameters: Increase the minimum confidence to see how it impacts the rules

min_confidence_adjusted = 0.5

rules_adjusted = association_rules(freq_itemsets, metric="confidence",
min_threshold=min_confidence_adjusted)

print(f"\nRules with minimum confidence of {min_confidence_adjusted}:")

print(rules_adjusted[['antecedents', 'consequents', 'support', 'confidence', 'lift']])
```

Lab Assignments

SET A

1. Case Study: As a mall owner, identify the most profitable customer groups based on Spending Score by applying hierarchical algorithms.
   a. Apply Data pre-processing (Label Encoding, Data Transformation….) techniques if necessary.
   b. Perform data-preparation (Train-Test Split)
   c. Apply hierarchical clustering Machine Learning Algorithm
   d. Evaluate Model.
   Download the Mall Customers Dataset from Kaggle. Data Set: https://www.kaggle.com/shwetabh123/mall-customers
2. Case Study: As a mall owner, identify the most profitable customer groups based on Spending Score by applying k- means algorithms.
   a) Apply Data pre-processing (Label Encoding, Data Transformation….) techniques if necessary.
   b) Perform data-preparation (Train-Test Split)
   c) Apply K- means clustering Machine Learning Algorithm
   d) Evaluate Model.
   Download the Mall Customers Dataset from Kaggle. Data Set: https://www.kaggle.com/shwetabh123/mall-customers

SET B

1. Write a Python Program to Implement Apriori Algorithm.Generate frequent itemsets using the Apriori algorithm.
   Dataset link: https://www.kaggle.com/datasets/hemanthkumar05/market-basket-optimization
   The dataset contains 7501 transactions listing items sold per transaction.
2. Write a Python Program to Implement Apriori Algorithm.Identify association rules between items using the Apriori algorithm.
   Download the Market Basket Optimization dataset
   The dataset contains 7501 transactions listing items sold per transaction.

SET C

1. Case Study: As a mall owner, identify the most profitable customer groups based on Spending Score by applying Agglomerative clustering algorithms.
    a) Apply Data pre-processing (Label Encoding, Data Transformation….) techniques if necessary.
    b) Perform data-preparation (Train-Test Split)
    c) Apply Agglomerative clustering Machine Learning Algorithm
    d) Evaluate Model.
2. Write a Python Program to Implement FP-growth algorithm to find frequent item sets and generate association rules.

Assignment Evaluation

0: Not Done [ ]	1: Incomplete [ ]	2: Late Complete [ ]
3:Need Improvement [ ]	4: Complete [ ]	5: Well Done [ ]

Signature of Teacher

References:

- ✓ https://realpython.com/linear-regression-in-python/
- ✓ MachineLearningBasics:SimpleLinearRegression|byGurucharanMK|TowardsDataScience
- ✓ LinearRegression(PythonImplementation)-GeeksforGeeks
- ✓ SolvingLinearRegressioninPython-GeeksforGeeks
- ✓ SimpleLinearRegressioninPython|SimpleLinearRegressionTutorial(analyticsvidhya.com)
- ✓ SimpleLinearRegression:APracticalImplementationinPython-AskPython
- ✓ https://datatofish.com/multiple-linear-regression-python/
- ✓ HowtoDetectandRemoveOutliers|OutlierDetectionAndRemoval(analyticsvidhya.com)
- ✓ OutlierDetection:AnIntroductionToItsTechniques(digitalvidya.com)
- ✓ TE_IT_ML_ASSIGNMENTS_SPPU/Assignment_5.ipynb at main · RanjeetKumbhar01/TE_IT_ML_ASSIGNMENTS_SPPU · GitHub
- ✓ https://www.datacamp.com/community/tutorials/svm-classification-scikit-learn-python
- ✓ https://www.javapoint.com
- ✓ https://www.tutorialspoint.com/data_mining/dm_classification_prediction.htm
- ✓ https://www.javatpoint.com/machine-learning-decision-tree-classification-algorithm
- ✓ https://www.analyticsvidhya.com/blog/2017/09/understaing-support-vector-machine-example-code/
- ✓ https://www.analyticsvidhya.com/blog/2017/09/understaing-support-vector-machine-example-code/
- ✓ https://github.com/Eligijus112/decision-tree-python
- ✓ GitHub - RanjeetKumbhar01/TE_IT_ML_ASSIGNMENTS_SPPU: 314448 : Laboratory Practice-I (Machine Learning) SPPU
- ✓ LP-I/Machine_Learning/Assignment_4/ML_Assignment_4_33209.ipynb at main · vaibhavbichave/LP-I · GitHub
- ✓ https://archive.ics.uci.edu/ml/datasets/Zoo